BUILDING A MODEL TO RATE SENTIMENTS OF TWEETS ABOUT GOOGLE AND APPLE PRODUCTS Student Name: Stephen Jilani Project: Phase 3 Date:14 June 2026

1. BUSINESS UNDERSTANDING

A Kenyan company an is in a predicament on classifying Twitter sentiments on Google and Apple products. This is with the aim of knowing the perceptions of the products in question to the public. The company has an objective to build a classification models that will help them classify sentiments into positive/negative(binary classification) or positive/neutral/negative(multiclass classification).

STAKEHOLDERS

The stakeholder of the project is the Kenyan company at large, however, the specific persons of interest include but not limited to the following:

```
1. Customer support.
2. Product teams.
3. Sales and marketing department.
4. Research and development department.
```

BUSINESS OBJECTIVES The primary objective is to build a classification model on twitter reviews on Google and Apple products to better understand customer perception and improve decision making.

```
1. Compare multiple models for binary sentiment classification.
2. Evaluate model performance using F1 Score, precision, and recall.
3. Provide insights on customer sentiment distribution.
```

BUSINESS QUESTIONS

```
1. What are the main drivers of sentiment classification.
2. What are the most common emotions expressed in reviews.
3. Which model performs best for sentiment analysis.
```

PROJECT SUCCESS METRICS

This project is going to be a success if the metrics below are met:

```
1. If it's possible to identify a binary model that yields the best
F1-score, precision and recall.
2. If it's possible that the multiclass classifier achieves an
acceptable F1-score, precision and recall.
```

2. DATA UNDERSTANDING

The data was obtained from CriwdFlower via data.world, and is composed of sentiments which were rated by human raters as positive, negative or either in over 9,000 tweets. The sentiments are in relation to Apple and Google products.

Preliminary analysis on the data for better understanding of the data.

```
#Importation of necesarry libraries
import pandas as pd
import numpy as np
import re

import matplotlib.pyplot as plt
import seaborn as sns

from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split

from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import MultinomialNB
from sklearn.svm import LinearSVC

from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    f1_score
)
```

```
C:\Users\USER\anaconda3\Lib\site-packages\pandas\core\computation\
expressions.py:23: UserWarning: Pandas requires version '2.10.2' or
newer of 'numexpr' (version '2.8.7' currently installed).
  from pandas.core.computation.check import NUMEXPR_INSTALLED
C:\Users\USER\anaconda3\Lib\site-packages\pandas\core\arrays\
masked.py:56: UserWarning: Pandas requires version '1.4.2' or newer of
'bottleneck' (version '1.3.7' currently installed).
  from pandas.core import (
```

```
#Loading the dataset
df = pd.read_csv('judge-1377884607_tweet_product_company.csv',
encoding="latin1")
df.head()
```

```
                                          tweet_text  \
0  .@wesley83 I have a 3G iPhone. After 3 hrs twe...
1  @jessedee Know about @fludapp ? Awesome iPad/i...
2  @swonderlin Can not wait for #iPad 2 also. The...
3  @sxsw I hope this year's festival isn't as cra...
4  @sxtxstate great stuff on Fri #SXSW: Marissa M...

  emotion_in_tweet_is_directed_at  \
0                          iPhone
1              iPad or iPhone App
```

```
2                                iPad
3             iPad or iPhone App
4                           Google

  is_there_an_emotion_directed_at_a_brand_or_product
0                                   Negative emotion
1                                   Positive emotion
2                                   Positive emotion
3                                   Negative emotion
4                                   Positive emotion

#Checking descriptive statistics on the data
df.info()

#Observations
#The data has 3 columns and 9,003 rows. The colums are for the tweet
text, the brand directed to and the sentiment classification. The
columns will need
# renamed to make their names shorter.
#All the 3 columns are formated in text

<class 'pandas.DataFrame'>
RangeIndex: 9093 entries, 0 to 9092
Data columns (total 3 columns):
 #   Column                                                Non-Null
Count  Dtype
---  ------
--------------  -----
 0   tweet_text                                            9092 non-null
str
 1   emotion_in_tweet_is_directed_at                       3291 non-null
str
 2   is_there_an_emotion_directed_at_a_brand_or_product  9093 non-null
str
dtypes: str(3)
memory usage: 1.4 MB

#Checking the columns on brands and sentiments to see the unique
entries there. The tweent column is expected to have unique
entries.However, we will
#check just to be sure.

print(df['emotion_in_tweet_is_directed_at'].unique())
print(df['emotion_in_tweet_is_directed_at'].value_counts())
print(df['is_there_an_emotion_directed_at_a_brand_or_product'].unique(
))
print(df['is_there_an_emotion_directed_at_a_brand_or_product'].value_c
ounts())

#Observations
```

```
#The "brand column has quite a number of unique entries but from the
results below. There is a possibility that the entries there can be
transformed into
#either Apple or Google. The same can also be said about the
"sentiment classification" column which has less unique items but all
seem to be able to
#be classified as either Negative, Positive or Neutral via data
cleaning.
```

```
<ArrowStringArray>
[                         'iPhone',              'iPad or iPhone App',
                            'iPad',                         'Google',
                               nan,                        'Android',
                           'Apple',                    'Android App',
 'Other Google product or service',  'Other Apple product or service']
Length: 10, dtype: str
emotion_in_tweet_is_directed_at
iPad                                946
Apple                               661
iPad or iPhone App                  470
Google                              430
iPhone                              297
Other Google product or service     293
Android App                          81
Android                              78
Other Apple product or service       35
Name: count, dtype: int64
<ArrowStringArray>
[                  'Negative emotion',                   'Positive
emotion',
 'No emotion toward brand or product',                        'I can't
tell']
Length: 4, dtype: str
is_there_an_emotion_directed_at_a_brand_or_product
No emotion toward brand or product    5389
Positive emotion                      2978
Negative emotion                       570
I can't tell                           156
Name: count, dtype: int64
```

3. DATA CLEANING AND EXPLORATORY DATA ANALYSIS

```
#Checking for blank columns in the dataset
df.isna().sum()

#Observation
#The tweet text has one blank row. This may have to be dropped as the
lack of text measn there is nothing to be rated.
#The brand column has over 50% of blank rows and this is significant
```

```
hence may need to be filled based on the condition of the tweet
#The emotion column has no  blank entries which is a good thing.
```

```
tweet_text                                                1
emotion_in_tweet_is_directed_at                        5802
is_there_an_emotion_directed_at_a_brand_or_product        0
dtype: int64
```

```
#Checking if the tweet text has any duplicates as this is expected to
have unique entries only.
df['tweet_text'].duplicated().sum()
#This observation is an indication of possibility that there are
reviews that were submitted twice. The possibility of having 2 people
submiting an indentical review
#on a product arevery minimal. This will be sorted out by dropping the
duplicates.
```

```
27
```

```
#Running the columns of the dataset to see them before renaming
df.columns
```

```
Index(['tweet_text', 'emotion_in_tweet_is_directed_at',
       'is_there_an_emotion_directed_at_a_brand_or_product'],
      dtype='str')
```

```
#Renaming the columns of the dataset and running the dataset to see if
the names were replaced
new_columns = ['Tweet_text','Brand', 'Classification']
df.columns = new_columns
df.head()
```

```
                                          Tweet_text
Brand  \
0  .@wesley83 I have a 3G iPhone. After 3 hrs twe...
iPhone
1  @jessedee Know about @fludapp ? Awesome iPad/i...  iPad or iPhone
App
2  @swonderlin Can not wait for #iPad 2 also. The...
iPad
3  @sxsw I hope this year's festival isn't as cra...  iPad or iPhone
App
4  @sxtxstate great stuff on Fri #SXSW: Marissa M...
Google

     Classification
0  Negative emotion
1  Positive emotion
2  Positive emotion
3  Negative emotion
4  Positive emotion
```

```
#The Tweet_text column has one blank row which will be dropped. We
will also remove any white spaces or empty strings if any.
df = df.dropna(subset =['Tweet_text'])
df = df[df['Tweet_text'].str.strip() !=""]

#Printing info for the Tweet_text column to see if the rows have been
dropped.
df['Tweet_text'].shape
#Observation
#The column initially had 9,003 rows but now 1 has been dropped.
```

```
(9092,)
```

```
# Dropping the duplicates in the tweet_text column. This is because
they might be reviews which have been entered more than once as its
almost
#impossible for2 people to give an identical review in form of a
tweet.

df = df.drop_duplicates(subset = ['Tweet_text'])
df['Tweet_text'].shape
```

```
(9065,)
```

```
#We will now go to the classification column and ensure it's cleaned
then we will go back to the tweet_text column later on.

df['Classification'].unique()

#The column has emotions like "I can't tell" and 'No emotion toward
brand or product' which are the ones that can be considered as
neutral.
#We will replace these strings with 'neutral"
```

```
<ArrowStringArray>
[                  'Negative emotion',                    'Positive
emotion',
 'No emotion toward brand or product',                         'I can't
tell']
Length: 4, dtype: str
```

```
df['Classification']= df['Classification'].replace({'No emotion toward
brand or product':'Neutral emotion', "I can't tell":'Neutral
emotion'})
df['Classification'].unique()
```

```
<ArrowStringArray>
['Negative emotion', 'Positive emotion', 'Neutral emotion']
Length: 3, dtype: str
```

```
#Checking if the tweet_text column contains keywords that can match to
a specific brand
```

```
df[df['Brand'].isna()]['Tweet_text'].head(20)

#Observation
#It can be noted that there are words like iPad, Google, App Store,
iTunes which can be used to determine which brand a tweet is
associated with.
```

```
5     @teachntech00 New iPad Apps For #SpeechTherapy...
16    Holler Gram for iPad on the iTunes App Store -...
32    Attn: All  #SXSW frineds, @mention Register fo...
33        Anyone at  #sxsw want to sell their old iPad?
34    Anyone at  #SXSW who bought the new iPad want ...
35    At #sxsw.  Oooh. RT @mention Google to Launch ...
37    SPIN Play - a new concept in music discovery f...
39    VatorNews - Google And Apple Force Print Media...
41    HootSuite - HootSuite Mobile for #SXSW ~ Updat...
42    Hey #SXSW - How long do you think it takes us ...
43    Mashable! - The iPad 2 Takes Over SXSW [VIDEO]...
44    For I-Pad ?RT @mention New #UberSocial for #iP...
46    Hand-Held ◗Û÷Hobo◗Ûª: Drafthouse launches ◗Û÷H...
48    Orly....? ◗ÛÏ@mention Google set to launch new...
50    Khoi Vinh (@mention says Conde Nast's headlong...
51    ◗ÛÏ@mention {link} &lt;-- HELP ME FORWARD THIS...
52    ◗÷¼ WHAT? ◗÷_ {link} ◗ã_ #edchat #musedchat #s...
53    .@mention @mention on the location-based 'fast...
54    ◗ÛÏ@mention @mention #Google Will Connect the ...
56    {link} RT @mention &quot;Google before you twe...
Name: Tweet_text, dtype: str
```

```
# For filling the null values in our brand column, we will lower the
text in the tweet_text column and use key words to fill the missing
values.
keywords_apple = [
    'apple', 'iphone', 'ipad', 'macbook', 'ios',
    'itunes', 'safari', 'icloud', 'ipod', 'app store'
]

keywords_google = [
    'google', 'android', 'gmail', 'youtube',
    'chrome', 'google maps', 'pixel', 'nexus'
]

def refer_brand(text):
    text = str(text).lower()

    apple = any(word in text for word in keywords_apple)
    google = any(word in text for word in keywords_google)

    if apple and google:
        return 'Both'
```

```
    elif apple:
        return 'Apple'
    elif google:
        return 'Google'
    else:
        return None

null_brands = df['Brand'].isna()
df.loc[null_brands, 'Brand'] = df.loc[null_brands,
'Tweet_text'].apply(refer_brand)

#Seeing if the code above has worked by checking how many null values
are after running the code above.
df['Brand'].isna().sum()

#Observation
#The null rowshave reduced from 5,000 to 700 meaning that we have
filled a bigger percentage of the null rows. Next we will see the
unique values in
#the column now to see if we have something to drop. It's essential
that we drop the remaining 700 null rows in the brand column.
```

```
702
```

```
#Checking unique entries in the brand column
df['Brand'].unique()

#Based on the output, there needs standardization in the brand column
so that we can only have google or apple as our unique entries
```

```
<ArrowStringArray>
[                         'iPhone',              'iPad or iPhone App',
                            'iPad',                          'Google',
                           'Apple',                         'Android',
                     'Android App', 'Other Google product or service',
                            'Both',                             nan,
  'Other Apple product or service']
Length: 11, dtype: str
```

```
#Ensuring that there is standardization in the column to Google and
Apple only

df['Brand']= df['Brand'].replace({'iPhone':'Apple', "iPad or iPhone
App":'Apple', 'iPad':'Apple','Android':'Google','Android
App':'Google','Other Google product or service':'Google','Other Apple
product or service':'Apple'})
df['Brand'].unique()
#Observation
#The rows which have both do not direct us to any specific brand hence
do not add any relevance to our to be classification model. The same
```

```
applies to
#the null values even after filling them. These will be dropped.

<ArrowStringArray>
['Apple', 'Google', 'Both', nan]
Length: 4, dtype: str

#Dropping all the blank rows in the 'Brand' column and those that have
both Google and Apple as they are not relevant to our classificaition
problem.

df = df.dropna(subset = ['Brand'])
df= df[~df['Brand'].isin(['Both'])]
print(df['Brand'].unique())
print(df['Brand'].shape)


<ArrowStringArray>
['Apple', 'Google']
Length: 2, dtype: str
(8154,)
```

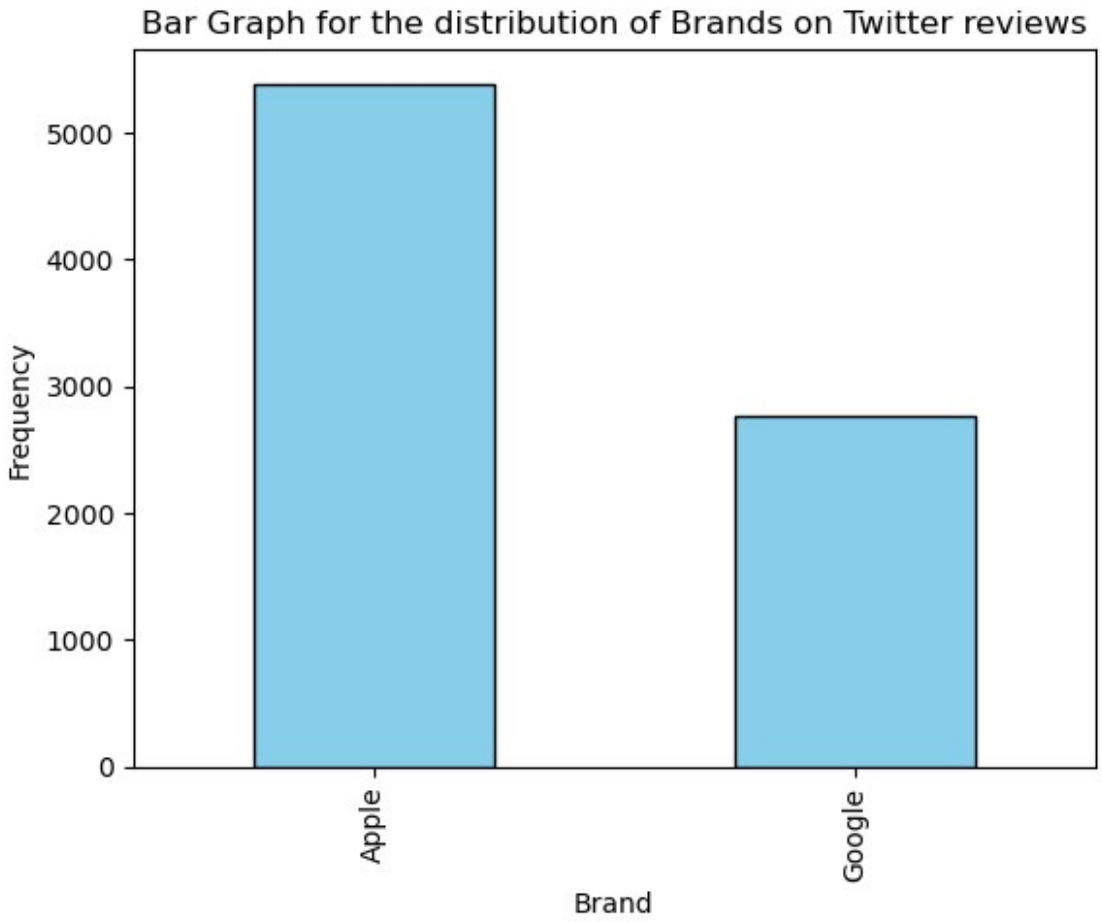
EXPLORATORY DATA ANALYSIS ON NON-TEXT COLUMNS The none-text columns here are the Beand and Classification columns

```
#Running the dataset for visibility
df.head()

                                          Tweet_text   Brand
Classification
0  .@wesley83 I have a 3G iPhone. After 3 hrs twe...   Apple  Negative
emotion
1  @jessedee Know about @fludapp ? Awesome iPad/i...   Apple  Positive
emotion
2  @swonderlin Can not wait for #iPad 2 also. The...   Apple  Positive
emotion
3  @sxsw I hope this year's festival isn't as cra...   Apple  Negative
emotion
4  @sxtxstate great stuff on Fri #SXSW: Marissa M...  Google  Positive
emotion
```

UNIVARIATE ANALYSIS ON NON-TEXT COLUMNS

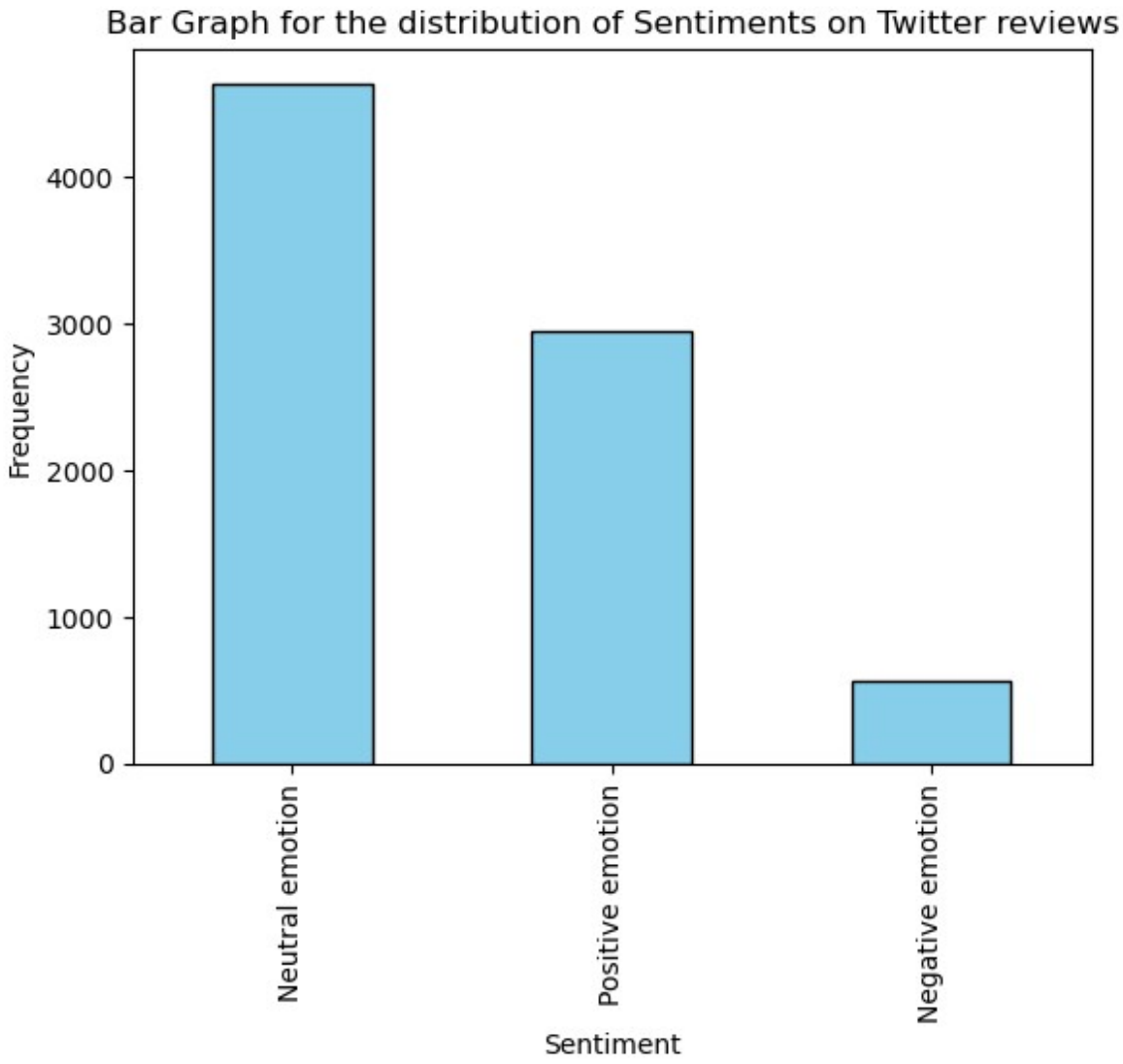
```
#Plotting a bar graph to show the frequency distribution between
Google and Apple brands in the Brand column

brand_names = df['Brand'].value_counts()

brand_names.plot(kind = 'bar', color= 'skyblue', edgecolor = 'black')
plt.xlabel('Brand')
```

```
plt.ylabel('Frequency')
plt.title('Bar Graph for the distribution of Brands on Twitter
reviews')
plt.show()
```



The bar graph shows that Apple products get more revieews compared to the Google products. This may lead to a bias problem in the model. However, a ratio of less that 1:2 is manageable.

```
#Plotting a bar graph to show the frequency distribution between
Negative, Positive and Neutral reviews

sentiment_class = df['Classification'].value_counts()

sentiment_class.plot(kind = 'bar', color= 'skyblue', edgecolor =
'black')
plt.xlabel('Sentiment')
plt.ylabel('Frequency')
plt.title('Bar Graph for the distribution of Sentiments on Twitter
reviews')
plt.show()
```

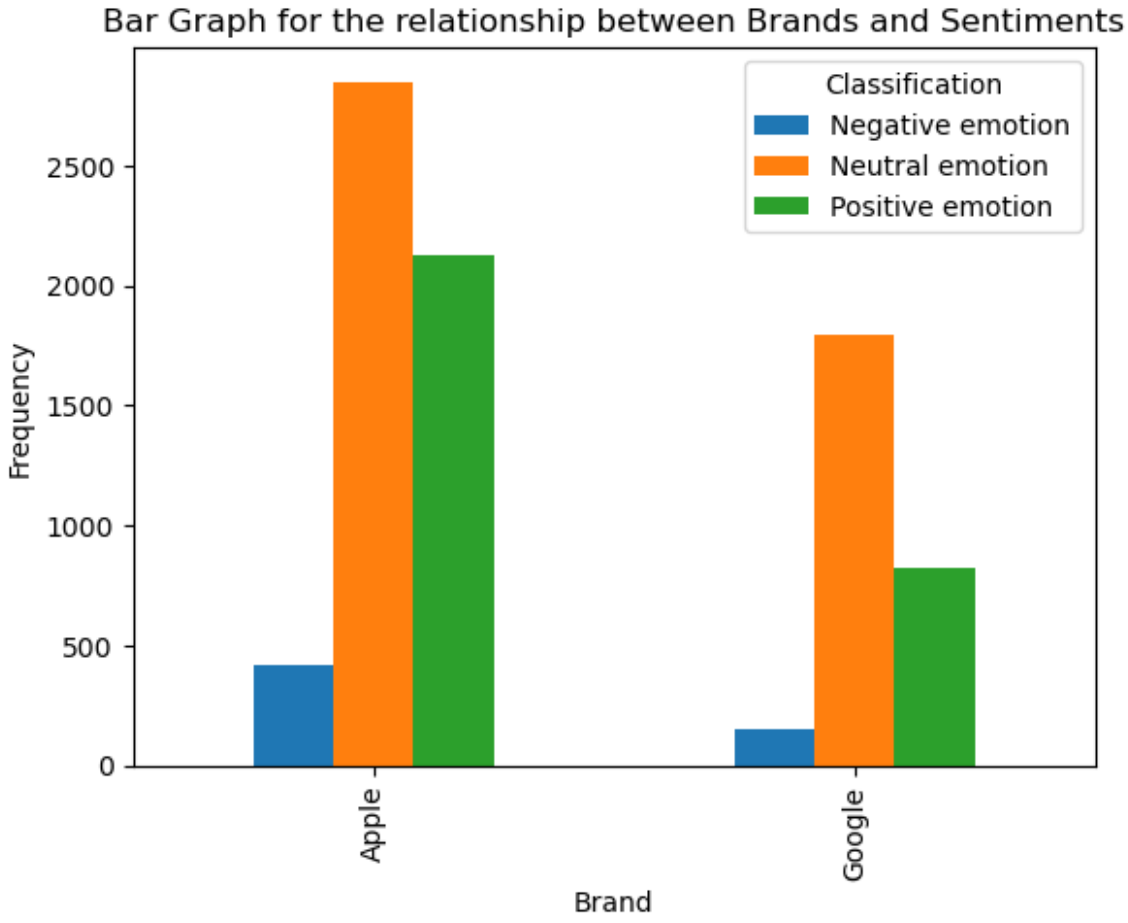


The bar graph shows that neutral emotions are the majority in the dataset, followed by positive and then negative emotions. This may be an indication that Twitter users are quite indifferent when it comes to choosing either Google or Apple products.

BIVARIATE ANALYSIS ON NON-TEXT COLUMNS

```
#Plotting a clustered bar graph on the Brand and Classification column

brand_class = pd.crosstab(
    df['Brand'],
    df['Classification']
)

brand_class.plot(kind = 'bar')#, figsize=(8,5), edgecolor='black')
plt.xlabel('Brand')
plt.ylabel('Frequency')
plt.title('Bar Graph for the relationship between Brands and
```

```
Sentiments')
plt.show()
```



The graph above shows that Apple products are more talked about compared to google products. This is evident in Apple products having more frequency in the dataset. However, the most sentiments are the neutral ones meaning that most reviewers are indifferent on the products they use.

Negative emotions are generally the lowest across the dataset as well.

TEXT PREPROCESSING This section is meant to prepare the tweet_text column for analysis and modelling

```
#Importing NLP libraries not already imported above
import nltk
import string

nltk.download('punkt') # for tokenization
nltk.download('stopwords') # for removing stopwords
nltk.download('wordnet')   # for lemmatization
```

```
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\USER\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to
[nltk_data]     C:\Users\USER\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to
[nltk_data]     C:\Users\USER\AppData\Roaming\nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
```

```
True
```

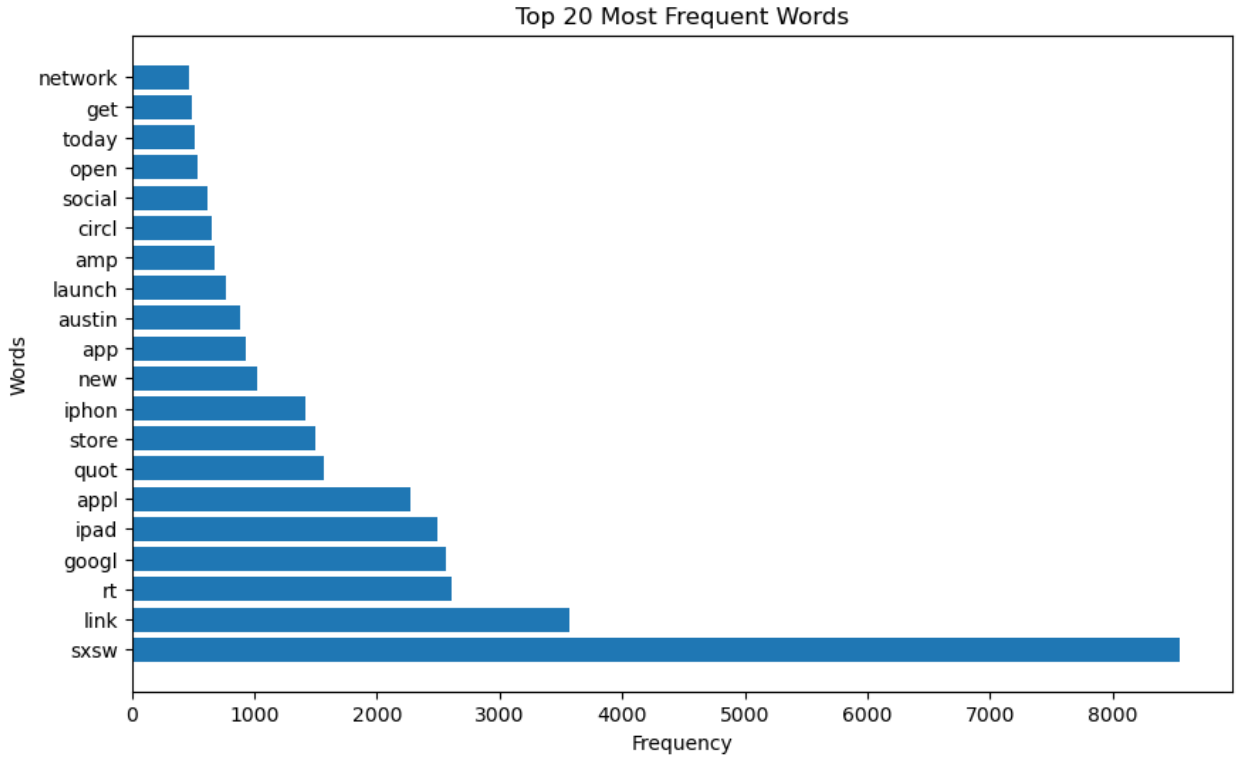
```
from nltk.stem import PorterStemmer

#Cleaning and preprocessing of the tweet_text column.

def preprocess(text):
    #lowercase
    text = text.lower()
    #removing mentions
    text =re.sub(r'[@@]\s*\w+', '', text)
    #removing URLs
    text = re.sub(r'http\S+', '', text)
    #tokenization
    tokens = nltk.word_tokenize(text)
    #removing punctuaton and numbers
    tokens = (word for word in tokens if word.isalpha())
    #removing stopwords
    stop_words = set(stopwords.words('english'))
    tokens = (word for word in tokens if word not in stop_words)
    #lematization
    lemma = WordNetLemmatizer()
    tokens = [lemma.lemmatize(word) for word in tokens]
    #stemming
    stemma = PorterStemmer()
    tokens = [stemma.stem(word) for word in tokens]
    return ' '.join(tokens)

#Applying the results of the code above into our original dataset df

df['clean_text'] = df['Tweet_text'].apply(preprocess)
df.head()
```

	Tweet_text	Brand	Classification	clean_text
0	.@wesley83 I have a 3G iPhone. After 3 hrs twe...	Apple		
1	@jessedee Know about @fludapp ? Awesome iPad/i...	Apple		
2	@swonderlin Can not wait for #iPad 2 also. The...	Apple		
3	@sxsw I hope this year's festival isn't as cra...	Apple		
4	@sxtxstate great stuff on Fri #SXSW: Marissa M...	Google		

```
0  Negative emotion  iphon hr tweet dead need upgrad plugin station...
1  Positive emotion  know awesom app like appreci design also give ...
2  Positive emotion                          wait ipad also sale sxsw
3  Negative emotion        hope year festiv crashi year iphon app sxsw
4  Positive emotion  great stuff fri sxsw marissa mayer googl tim t...
```

EXPLORATORY DATA ANALYSIS ON TEXT COLUMN

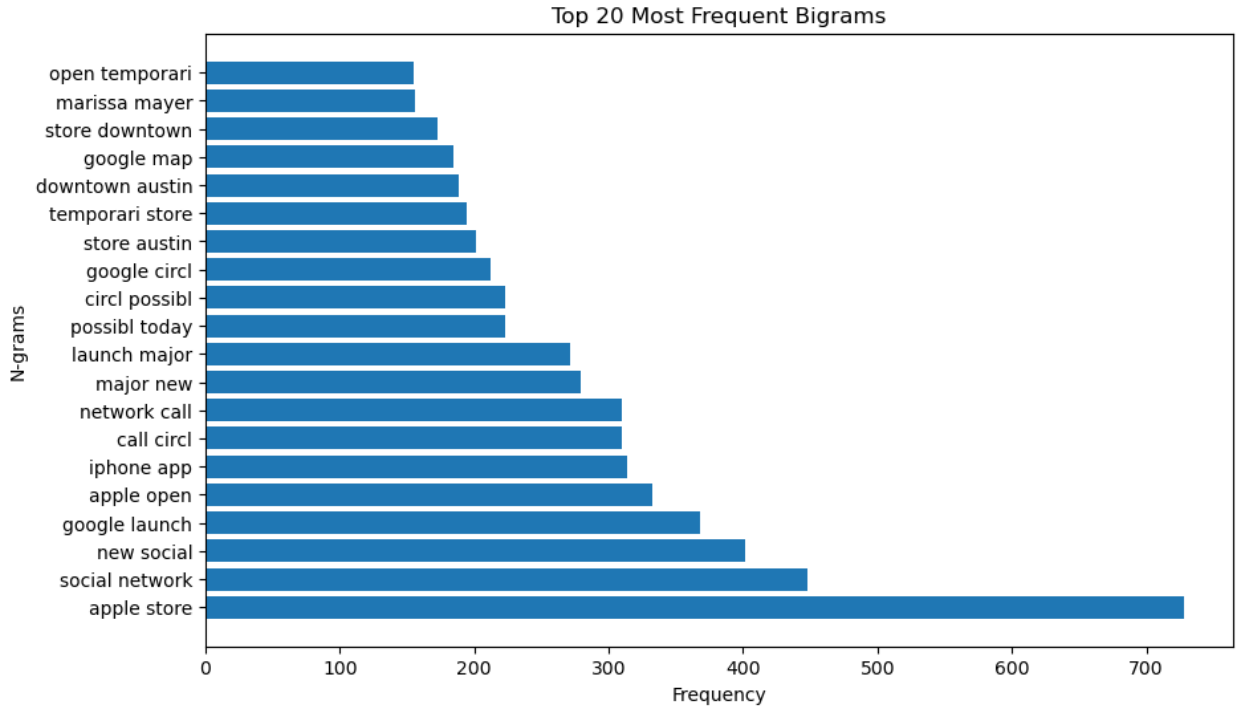
```
#Checking for frequent words and vizualiting them as well.
#Importing the counter

from collections import Counter

frequent_words = " ".join(df['clean_text']).split()
word_counts = Counter(frequent_words).most_common(20)

word_df = pd.DataFrame(word_counts, columns=['word', 'count'])

#The dataset contains a lot of social media noisy text like 'rt',
'link', 'sxsw' etc which adds noise in our dataset and may affect how
our model learns.
#We will create a custom stop words list and remove these words
#The stemming applied was also aggressive such that words like 'Apple'
have been reduced to 'Appl' as seen below or google reduced to
'googl'. These need
#to be restored.

#Vizualizing the results of the frequent words
import matplotlib.pyplot as plt

plt.figure(figsize=(10,6))

plt.barh(word_df['word'], word_df['count'])

plt.xlabel('Frequency')
plt.ylabel('Words')
plt.title('Top 20 Most Frequent Words')

Text(0.5, 1.0, 'Top 20 Most Frequent Words')
```




```
#creating a custom stop words list
twitter_noise = ['rt', 'link', 'amp', 'quot', 'sxsw']

#Removing the custom stop words
df['clean_text'] = df['clean_text'].apply(
    lambda x: " ".join([word for word in x.split() if word not in
twitter_noise])
)

#Restoring apple key words that have been destroyed by stemming
df['clean_text'] = df['clean_text'].str.replace('appl', 'apple',
regex=False)
df['clean_text'] = df['clean_text'].str.replace('iphon', 'iphone',
regex=False)
df['clean_text'] = df['clean_text'].str.replace('googl', 'google',
regex=False)

print(df['clean_text'].str.contains('googl').sum())
print(df['clean_text'].str.contains('iphon').sum())
print(df['clean_text'].str.contains('appl').sum())

#The number of times these words appear has reduced meaning that they
have been replaced
```

```
2370
1383
2092
```

```
#Using N-grams to capture phrases and not single words
from sklearn.feature_extraction.text import CountVectorizer

vectorizer = CountVectorizer(ngram_range=(2,2), max_features=20)
phrases = vectorizer.fit_transform(df['clean_text'])

ngrams = (vectorizer.get_feature_names_out())
counts = phrases.toarray().sum(axis=0)

ngram_df = pd.DataFrame({
    'ngram':ngrams,
    'count':counts
}).sort_values(by='count',ascending=False)

#vizualizing the contents of the most common phrases

plt.figure(figsize=(10,6))

plt.barh(
    ngram_df['ngram'],
    ngram_df['count']
)

plt.xlabel('Frequency')
plt.ylabel('N-grams')
plt.title('Top 20 Most Frequent Bigrams')
plt.show()
```



4. DATA PREPARATION

The intention is to do binary and multiclass classifiers. Therefore, we will save 2 datasets; one for the binary classification and the other one for the multiclass classifier.

```
df['Classification'].unique()
```

```
<ArrowStringArray>
['Negative emotion', 'Positive emotion', 'Neutral emotion']
Length: 3, dtype: str
```

```
#Dropping the neutral emotion so that we can have only the positive
and negative emotions in the column and viewing the results. Thisis
going to be the dataset
#for binary classification
df_binary = df[df['Classification'] != 'Neutral emotion'].copy()
print(df_binary['Classification'].unique())
print(df_binary.head())
```

```
<ArrowStringArray>
['Negative emotion', 'Positive emotion']
Length: 2, dtype: str
                                          Tweet_text   Brand  \
0  .@wesley83 I have a 3G iPhone. After 3 hrs twe...   Apple
1  @jessedee Know about @fludapp ? Awesome iPad/i...   Apple
2  @swonderlin Can not wait for #iPad 2 also. The...   Apple
3  @sxsw I hope this year's festival isn't as cra...   Apple
4  @sxtxstate great stuff on Fri #SXSW: Marissa M...  Google

     Classification                                         clean_text

0  Negative emotion    iphone hr tweet dead need upgrad plugin station

1  Positive emotion  know awesom app like appreci design also give ...

2  Positive emotion                                wait ipad also sale

3  Negative emotion            hope year festiv crashi year iphone app

4  Positive emotion  great stuff fri marissa mayer google tim tech ...
```

```
df_multi = df.copy()
df_multi.head()
```

```
#This is going to the dataset for multiclass classifier
```

```
                                          Tweet_text   Brand  \
0  .@wesley83 I have a 3G iPhone. After 3 hrs twe...   Apple
1  @jessedee Know about @fludapp ? Awesome iPad/i...   Apple
2  @swonderlin Can not wait for #iPad 2 also. The...   Apple
3  @sxsw I hope this year's festival isn't as cra...   Apple
```

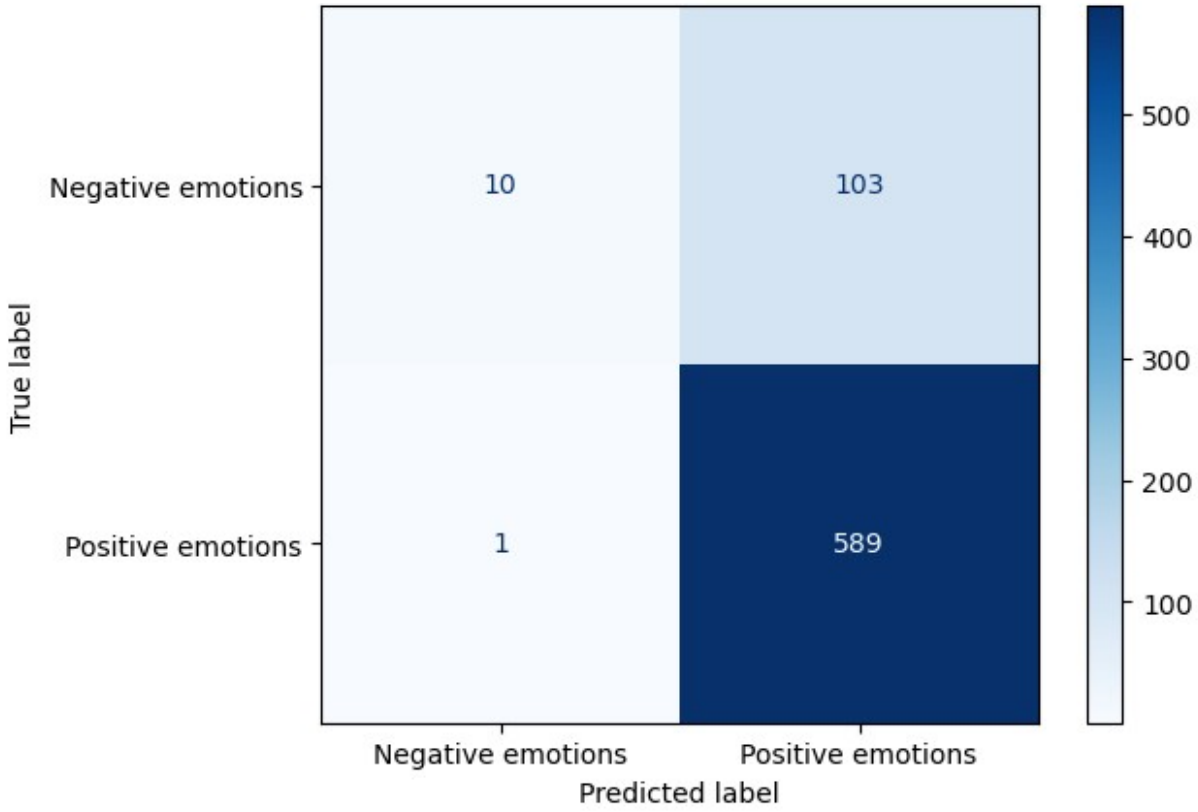
```
4  @sxtxstate great stuff on Fri #SXSW: Marissa M...  Google

     Classification                                          clean_text

0  Negative emotion    iphone hr tweet dead need upgrad plugin station

1  Positive emotion  know awesom app like appreci design also give ...

2  Positive emotion                                wait ipad also sale

3  Negative emotion            hope year festiv crashi year iphone app

4  Positive emotion  great stuff fri marissa mayer google tim tech ...
```

```
#Splitting my data for the binary classification into X and Y. The
feature is going to be the 'clean text' column and the target will be
the
#'classification'

X_train,X_test, y_train, y_test = train_test_split(
    df_binary['clean_text'],
    df_binary['Classification'],
    test_size = 0.2,
    random_state = 42,
    stratify = df_binary['Classification']
)

#Feature engineering using TF-IDF. This is to convert the text data
into numerical format
vectorize =TfidfVectorizer()
X_train_vec = vectorize.fit_transform(X_train)
X_test_vec = vectorize.transform(X_test)
```

5. MODELLING AND MODEL EVALUATION

```
#Logistic regression as the base model
log_model = LogisticRegression(max_iter=1000)
log_model.fit(X_train_vec, y_train)

LogisticRegression(max_iter=1000)

#Making predictions with the base model
y_pred_log = log_model.predict(X_test_vec)

#Model Evaluation
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

cm = confusion_matrix(y_test, y_pred_log)

display = ConfusionMatrixDisplay(confusion_matrix = cm,
```

```
display_labels=["Negative emotions", "Positive emotions"])
display.plot(cmap=plt.cm.Blues)
plt.show()

print(classification_report(y_test, y_pred_log))
```



	precision	recall	f1-score	support
Negative emotion	0.91	0.09	0.16	113
Positive emotion	0.85	1.00	0.92	590
accuracy			0.85	703
macro avg	0.88	0.54	0.54	703
weighted avg	0.86	0.85	0.80	703

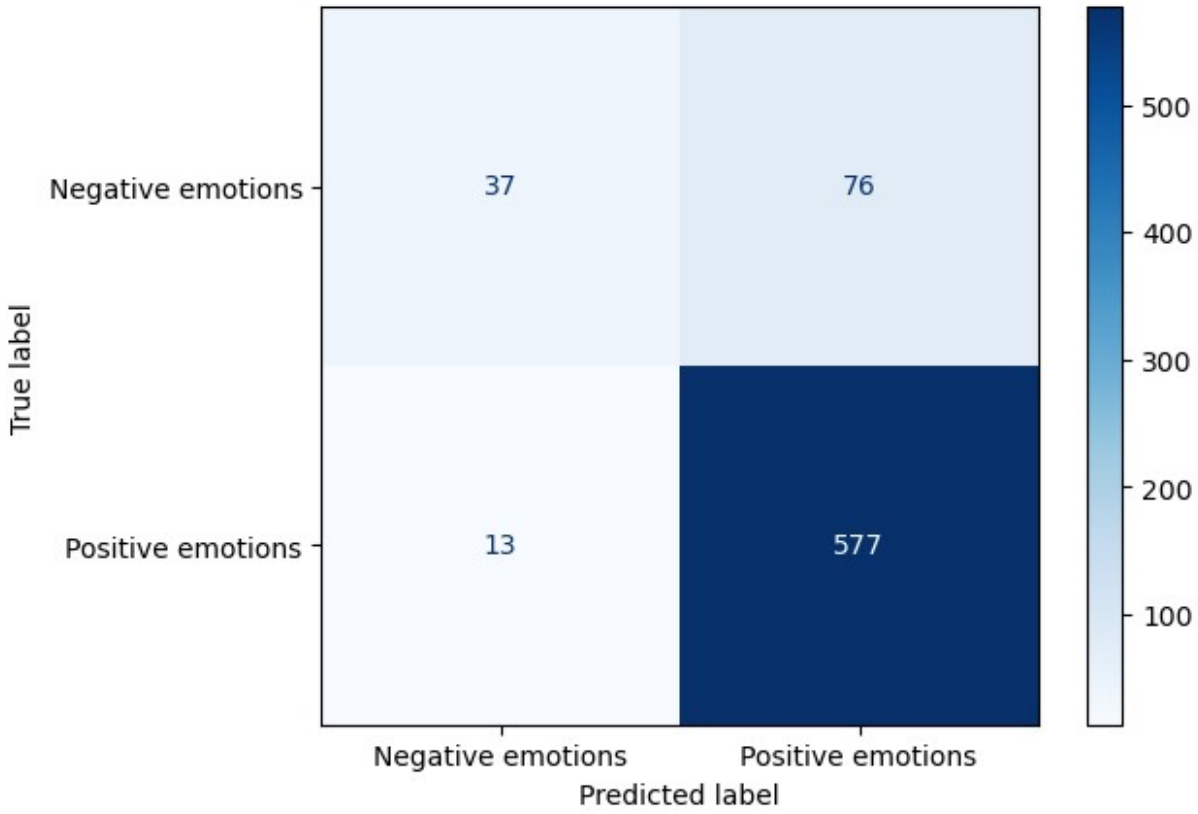
Confusion matrix interpretation

The model achieved a precision of 0.91, a recall of 0.09, f1-score of 0.16 on the negative emotions. The precision was 0.85, recall of 1.00 and f1 score of 0.92 for the positive emotions. This is however our base model and will be used for comparison purposes with the models explored below.

```
#Second model is the random forests

#Importation of libary
from sklearn.ensemble import RandomForestClassifier

#Fitting of the model

rand_forest = RandomForestClassifier(n_estimators=100, random_state=7)
rand_forest.fit(X_train_vec, y_train)
```

```
RandomForestClassifier(random_state=7)
```

```
#Making predictions using the Random Forest Model Above and model
evaluation

y_pred_rand_forest = rand_forest.predict(X_test_vec)

#Running confusion matrix on our model
cm_rand_forest = confusion_matrix(y_test, y_pred_rand_forest)

display_rand_forest = ConfusionMatrixDisplay(confusion_matrix =
cm_rand_forest, display_labels=["Negative emotions", "Positive
emotions"])
display_rand_forest.plot(cmap=plt.cm.Blues)
plt.show()
#Running the classification report for the model
print(classification_report(y_test, y_pred_rand_forest))
```



	precision	recall	f1-score	support
Negative emotion	0.74	0.33	0.45	113
Positive emotion	0.88	0.98	0.93	590
accuracy			0.87	703
macro avg	0.81	0.65	0.69	703
weighted avg	0.86	0.87	0.85	703

Confusion matrix interpretation

The RandomForest model led to improvements compared to the Logistic Regression model. This was due to the increase in recall from 0.09 to 0.33, F1-score from 0.16 to 0.45 for negative emotions. For positive emotions, the precision increased from 0.85 to 0.88, recall reduced from 1.0 to 0.98 and f1 score increased from 0.92 to 0.93

```
#Hyper parameter tuning for Random Forest

from sklearn.model_selection import GridSearchCV

#Instantiating the classifier
r_forest = RandomForestClassifier(random_state=7)
```

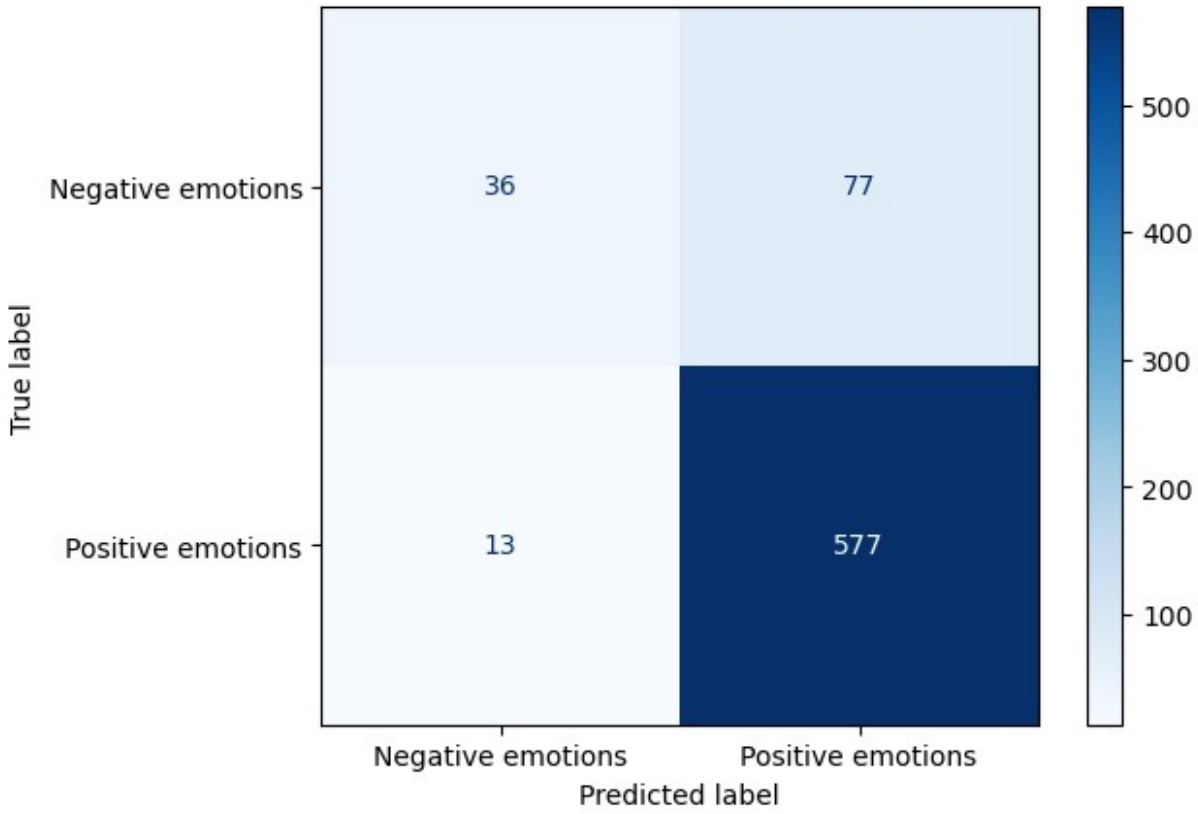
```
parameter_grid = {
    'n_estimators': [50, 100, 200, 300, 500]
}
#Fitting GridSearch

grid_search = GridSearchCV(
    estimator=r_forest,
    param_grid=parameter_grid,
    cv=5,
    scoring='f1_weighted',
    n_jobs=-1
)

grid_search.fit(X_train_vec, y_train)
```

```
GridSearchCV(cv=5, estimator=RandomForestClassifier(random_state=7),
n_jobs=-1,
             param_grid={'n_estimators': [50, 100, 200, 300, 500]},
             scoring='f1_weighted')
```

```
#Printing the results of hyperparameter tuning
print("Best parameters:", grid_search.best_params_)
print("Best CV score:", grid_search.best_score_)
```

```
Best parameters: {'n_estimators': 300}
Best CV score: 0.8491803956008257
```

```
#Evaluation of the best model

best_rand_forest = grid_search.best_estimator_

y_pred_rand_forest_tuned = best_rand_forest.predict(X_test_vec)


#Running confusion matrix and classification report
cm_rand_forest_tuned = confusion_matrix(y_test,
y_pred_rand_forest_tuned)

display_rand_forest_tuned = ConfusionMatrixDisplay(confusion_matrix =
cm_rand_forest_tuned, display_labels=["Negative emotions", "Positive
emotions"])
display_rand_forest_tuned.plot(cmap=plt.cm.Blues)
plt.show()
print(classification_report(y_test, y_pred_rand_forest_tuned))
```



	precision	recall	f1-score	support
Negative emotion	0.73	0.32	0.44	113
Positive emotion	0.88	0.98	0.93	590
accuracy			0.87	703
macro avg	0.81	0.65	0.69	703
weighted avg	0.86	0.87	0.85	703

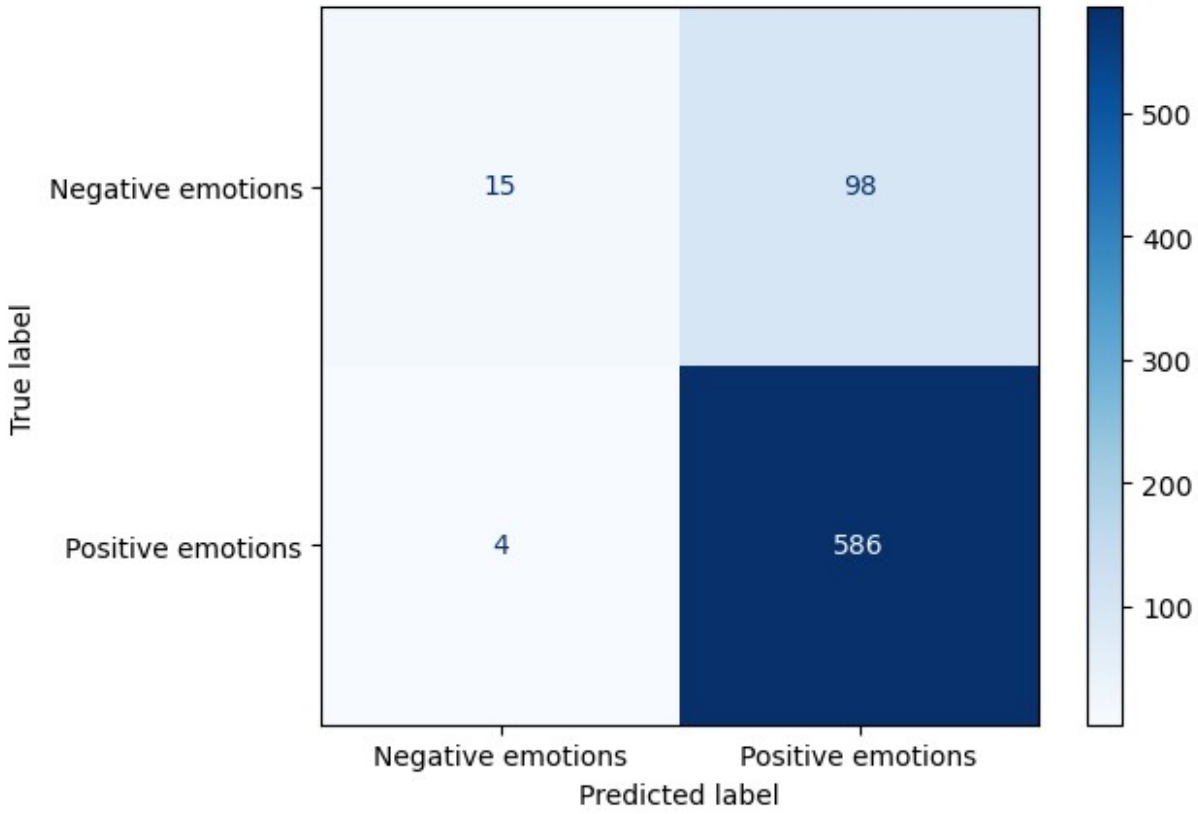
Confusion matrix interpretation

The model performance on the tuned Random Forest reduced significantly, this is to say that the base model RandomForest was already at a good step in balancing between bias and variance in the dataset, such that tuning just moved the model away from this balance.

```
#Installing XGBoost

!pip install xgboost
```

```
Requirement already satisfied: xgboost in c:\users\user\anaconda3\lib\
site-packages (3.2.0)
Requirement already satisfied: numpy in c:\users\user\anaconda3\lib\
site-packages (from xgboost) (1.26.4)
```

```
Requirement already satisfied: scipy in c:\users\user\anaconda3\lib\
site-packages (from xgboost) (1.17.1)

#Third model XG Boost

#Importation of libraries amd fitting of the model
import xgboost as xgb
from sklearn.preprocessing import LabelEncoder
from xgboost import XGBClassifier

label_encoder = LabelEncoder()

y_train_encoded = label_encoder.fit_transform(y_train)
y_test_encoded = label_encoder.transform(y_test)

model_XGBoost = xgb.XGBClassifier(n_estimators=100, learning_rate=0.1,
max_depth=5)
model_XGBoost.fit(X_train_vec, y_train_encoded)

XGBClassifier(base_score=None, booster=None, callbacks=None,
              colsample_bylevel=None, colsample_bynode=None,
              colsample_bytree=None, device=None,
early_stopping_rounds=None,
              enable_categorical=False, eval_metric=None,
feature_types=None,
              feature_weights=None, gamma=None, grow_policy=None,
              importance_type=None, interaction_constraints=None,
              learning_rate=0.1, max_bin=None, max_cat_threshold=None,
              max_cat_to_onehot=None, max_delta_step=None,
max_depth=5,
              max_leaves=None, min_child_weight=None, missing=nan,
              monotone_constraints=None, multi_strategy=None,
n_estimators=100,
              n_jobs=None, num_parallel_tree=None, ...)

# Making predictions and evaluation of the model

y_pred_XGBoost = model_XGBoost.predict(X_test_vec)


cm_XGBoost = confusion_matrix(y_test_encoded, y_pred_XGBoost)

display_XG_Boost = ConfusionMatrixDisplay(confusion_matrix =
cm_XGBoost, display_labels=["Negative emotions", "Positive emotions"])
display_XG_Boost.plot(cmap=plt.cm.Blues)
plt.show()
#Running the classification report for the model
print(classification_report(y_test_encoded, y_pred_XGBoost))
```



	precision	recall	f1-score	support
0	0.79	0.13	0.23	113
1	0.86	0.99	0.92	590
accuracy			0.85	703
macro avg	0.82	0.56	0.57	703
weighted avg	0.85	0.85	0.81	703

Confusion matrix interpretation

This model will be compared to the base RandomForest model as the tuned model took the model off-balance. This model has generally led to improvements in the precision from 0.74 to 0.79, the recall rate has reduced from 0.33 to 0.13. However, the f1 score has increased from 0.93 to 0.93. This is a considerable improvement hence making XG Boost the best model so far.

```
#Hyper paramater tuning for XGBoost

#Instantiating the classifier
xgb_tuned = XGBClassifier(
    random_state=42,
    use_label_encoder=False,
    eval_metric='logloss'
```

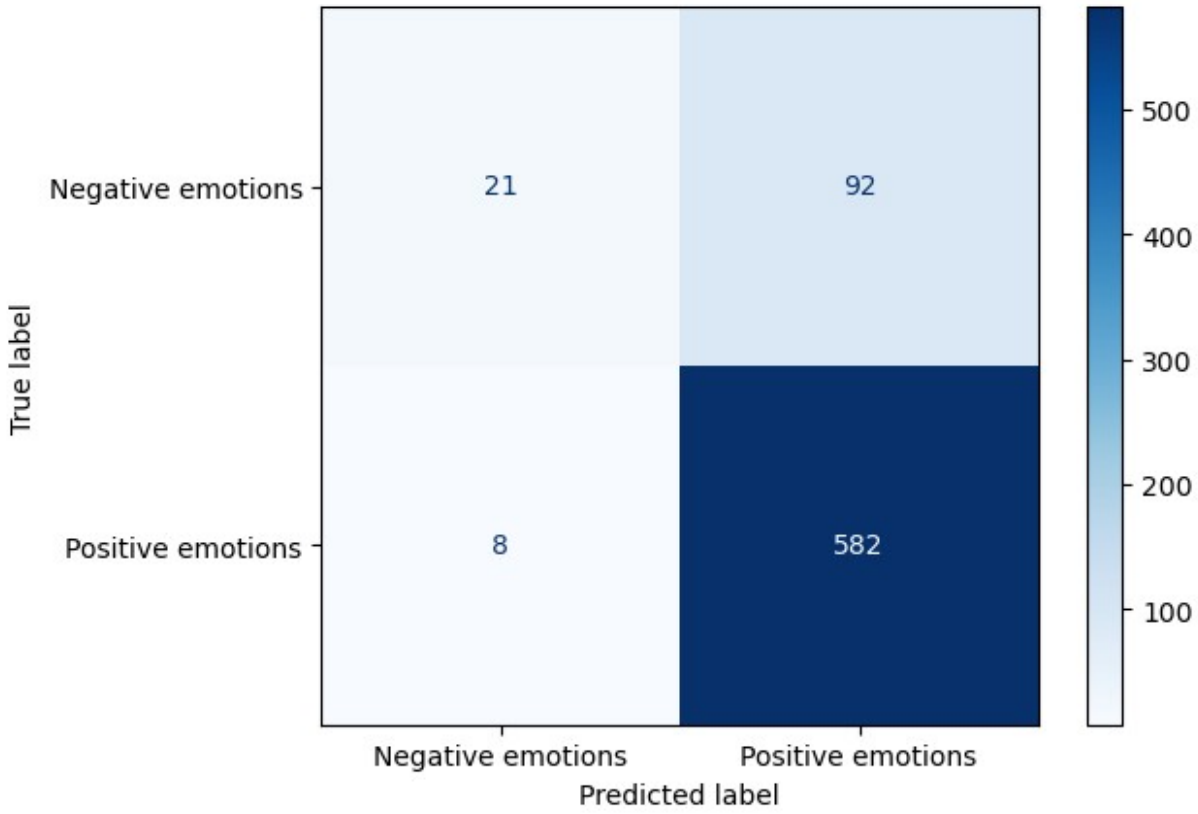
```
)

#Listing of parameters
parameter_grid_XG = {
    'n_estimators': [100, 200],
    'max_depth': [3, 5],
    'learning_rate': [0.1],
}

#Instantiating GridSearch
grid_search_XG = GridSearchCV(
    estimator=xgb_tuned,
    param_grid=parameter_grid_XG,
    cv=5,
    scoring='f1_weighted',
    n_jobs=-1,
    verbose=2
)

#Fitting the model
grid_search_XG.fit(X_train_vec, y_train_encoded)
```

```
Fitting 5 folds for each of 4 candidates, totalling 20 fits

C:\Users\USER\anaconda3\Lib\site-packages\xgboost\training.py:200:
UserWarning: [20:36:08] WARNING: C:\actions-runner\_work\xgboost\
xgboost\src\learner.cc:782:
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)

GridSearchCV(cv=5,
             estimator=XGBClassifier(base_score=None, booster=None,
                                     callbacks=None,
colsample_bylevel=None,
                                     colsample_bynode=None,
                                     colsample_bytree=None,
device=None,
                                     early_stopping_rounds=None,
                                     enable_categorical=False,
                                     eval_metric='logloss',
feature_types=None,
                                     feature_weights=None, gamma=None,
                                     grow_policy=None,
importance_type=None,
                                     interaction_constraint...
                                     max_cat_threshold=None,
                                     max_cat_to_onehot=None,
                                     max_delta_step=None,
max_depth=None,
```

```
                                        max_leaves=None,
min_child_weight=None,
                                        missing=nan,
monotone_constraints=None,
                                        multi_strategy=None,
n_estimators=None,
                                        n_jobs=None,
num_parallel_tree=None, ...),
             n_jobs=-1,
             param_grid={'learning_rate': [0.1], 'max_depth': [3, 5],
                         'n_estimators': [100, 200]},
             scoring='f1_weighted', verbose=2)

#Printing the best parameters for the model

print("Best Parameters:", grid_search_XG.best_params_)
print("Best CV Score:", grid_search_XG.best_score_)
```

```
Best Parameters: {'learning_rate': 0.1, 'max_depth': 5,
'n_estimators': 200}
Best CV Score: 0.8298656188696908
```

```
#Making predictions with the best parameters and evaluating the model.

best_XG = grid_search_XG.best_estimator_

y_pred_XG_tuned = best_XG.predict(X_test_vec)

cm_XG_tuned = confusion_matrix(y_test_encoded,y_pred_XG_tuned)

display_XG_tuned = ConfusionMatrixDisplay(confusion_matrix =
cm_XG_tuned, display_labels=["Negative emotions", "Positive
emotions"])
display_XG_tuned.plot(cmap=plt.cm.Blues)
plt.show()
print(classification_report(y_test_encoded, y_pred_XG_tuned))
```



	precision	recall	f1-score	support
0	0.72	0.19	0.30	113
1	0.86	0.99	0.92	590
accuracy			0.86	703
macro avg	0.79	0.59	0.61	703
weighted avg	0.84	0.86	0.82	703

Confusion matrix interpretation

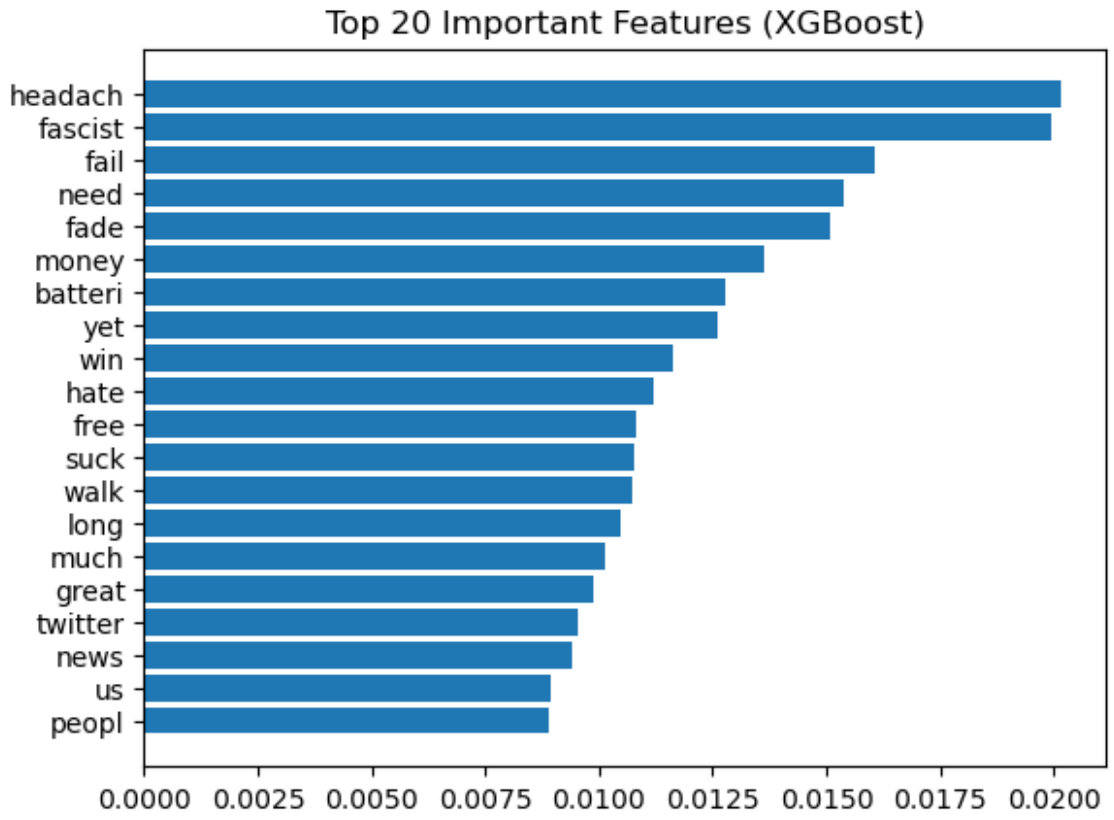
The tuned XG Boost has led to reduction in precision from 0.79 to 0.72, recall has improved from 0.13 to 0.19 while f1 score has also increased from 0.23 to 0.30 for negative emotions. For positive emotions, the precision, recall and f1 score have not changed at 0.86, 0.99 and 0.92 respectively. However, 2 out of the 3 metrics above have improved hence the tuned XG Boost model becomes the best out of all binary classification models explored.

```
#Model interpretability for XG Boost

feature_importance =pd.DataFrame({
    'feature':vectorize.get_feature_names_out(),
    'importance': best_XG.feature_importances_
}).sort_values('importance', ascending =False)
```

```
#Viewing the most important words
feature_importance.head(20)
```

	feature	importance
1540	headach	0.020152
1179	fascist	0.019959
1160	fail	0.016069
2241	need	0.015368
1159	fade	0.015062
2178	money	0.013630
283	batteri	0.012775
3820	yet	0.012619
3728	win	0.011640
1529	hate	0.011214
1301	free	0.010825
3217	suck	0.010770
3664	walk	0.010725
1977	long	0.010480
2205	much	0.010134
1457	great	0.009871
3529	twitter	0.009522
2260	news	0.009412
3594	us	0.008957
2464	peopl	0.008879

```
#Vizualization of feature importance above

top_features = feature_importance.head(20)

plt.barh(top_features["feature"], top_features["importance"])
plt.gca().invert_yaxis()
plt.title("Top 20 Important Features (XGBoost)")
plt.show()
```



Interpretation of key features

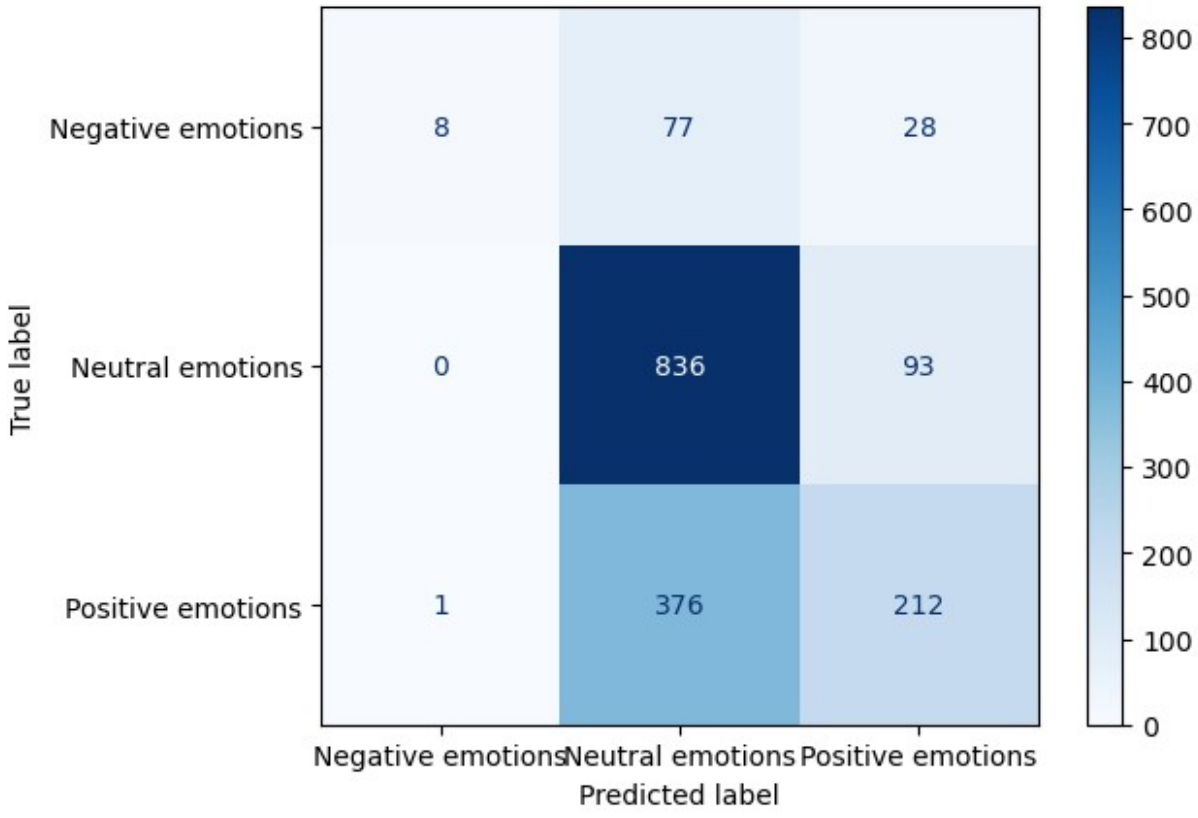
There are words like 'headache', 'fail', 'hate', 'suck' which are the biggets drivers of negative emotion. On the other hand, there are words like 'win', 'great', 'free' which are the bigets drivers of positive emotion. There are words that do not reallly indicate any sentiment like battery, twitter etc which cannot be traced to a particular sentiment.

```
#Running the multiclass dataset for visibility
df_multi.head()
```

```
                                          Tweet_text   Brand  \
0  .@wesley83 I have a 3G iPhone. After 3 hrs twe...   Apple
1  @jessedee Know about @fludapp ? Awesome iPad/i...   Apple
2  @swonderlin Can not wait for #iPad 2 also. The...   Apple
3  @sxsw I hope this year's festival isn't as cra...   Apple
4  @sxtxstate great stuff on Fri #SXSW: Marissa M...  Google

     Classification                                         clean_text

0  Negative emotion    iphone hr tweet dead need upgrad plugin station

1  Positive emotion  know awesom app like appreci design also give ...

2  Positive emotion                                wait ipad also sale
```

```
3  Negative emotion            hope year festiv crashi year iphone app

4  Positive emotion  great stuff fri marissa mayer google tim tech ...


#Multiclass Classifier Using XGBoost

#Splitting my data for the multiclass classification into X and Y. The
feature is going to be the 'clean text' column and the target will be
the
#'classification'

Xm_train,Xm_test, ym_train, ym_test = train_test_split(
    df_multi['clean_text'],
    df_multi['Classification'],
    test_size = 0.2,
    random_state = 42,
    stratify = df_multi['Classification']
)

#Feature engineering using TF-IDF. This is to convert the text data
into numerical format
vectorize_multi =TfidfVectorizer()
Xm_train_vec_multi = vectorize_multi.fit_transform(Xm_train)
Xm_test_vec_multi = vectorize_multi.transform(Xm_test)

#Fitting the multiclass model

label_encoder_multi = LabelEncoder()

ym_train_encoded_multi = label_encoder_multi.fit_transform(ym_train)
ym_test_encoded_multi = label_encoder_multi.transform(ym_test)

model_XGBoost_multi = xgb.XGBClassifier(n_estimators=100,
learning_rate=0.1, max_depth=5)
model_XGBoost_multi.fit(Xm_train_vec_multi, ym_train_encoded_multi)

XGBClassifier(base_score=None, booster=None, callbacks=None,
              colsample_bylevel=None, colsample_bynode=None,
              colsample_bytree=None, device=None,
early_stopping_rounds=None,
              enable_categorical=False, eval_metric=None,
feature_types=None,
              feature_weights=None, gamma=None, grow_policy=None,
              importance_type=None, interaction_constraints=None,
              learning_rate=0.1, max_bin=None, max_cat_threshold=None,
              max_cat_to_onehot=None, max_delta_step=None,
max_depth=5,
              max_leaves=None, min_child_weight=None, missing=nan,
              monotone_constraints=None, multi_strategy=None,
```

```
n_estimators=100,
              n_jobs=None, num_parallel_tree=None, ...)

#Making predictions and evaluation
y_pred_XGBoost_multi = model_XGBoost_multi.predict(Xm_test_vec_multi)


cm_XGBoost_multi = confusion_matrix(ym_test_encoded_multi,
y_pred_XGBoost_multi)

display_XG_Boost_multi = ConfusionMatrixDisplay(confusion_matrix =
cm_XGBoost_multi, display_labels=["Negative emotions","Neutral
emotions", "Positive emotions"])
display_XG_Boost_multi.plot(cmap=plt.cm.Blues)
plt.show()
#Running the classification report for the model
print(classification_report(ym_test_encoded_multi,
y_pred_XGBoost_multi))
```




```
              precision    recall  f1-score   support

           0       0.89      0.07      0.13       113
           1       0.65      0.90      0.75       929
           2       0.64      0.36      0.46       589
```

```
    accuracy                           0.65      1631
   macro avg       0.72      0.44      0.45      1631
weighted avg       0.66      0.65      0.60      1631
```

Confusion matrix interpretation

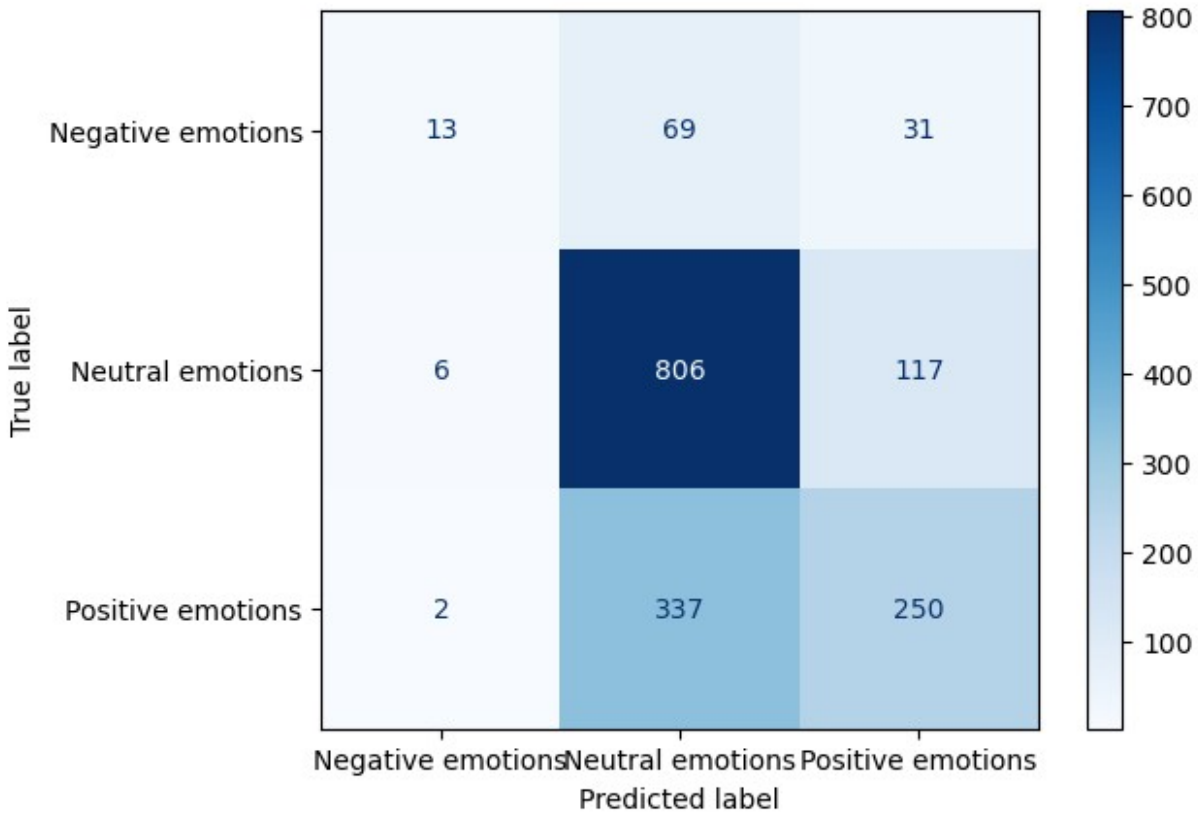
The model achieved precision of 0.89, 0.65 and 0.64 on Negative, Neutral and Positive emotions respectively; recall of 0.07, 0.90 and 0.36 respectively and f1-score of 0.13, 0.75 and 0.46 respectively. This is going to be the basis for comparison with the tuned model below to choose which is the best for the multiclass classification.

```
#Hyper paramater tuning for XGBoost for Multiclass Classification

#Instantiate the classifier
xgb_multi_tuned = XGBClassifier(
    random_state=42,
    use_label_encoder=False,
    eval_metric='logloss'
)

#Listing parameters
parameter_grid_XG_multi = {
    'n_estimators': [100, 200],
    'max_depth': [3, 5],
    'learning_rate': [0.1],
}

#Instantiating GridSearch
grid_search_XG_multi = GridSearchCV(
    estimator=xgb_multi_tuned,
    param_grid=parameter_grid_XG_multi,
    cv=5,
    scoring='f1_weighted',
    n_jobs=-1,
    verbose=2
)

#Fitting the model
grid_search_XG_multi.fit(Xm_train_vec_multi, ym_train_encoded_multi)
```

```
Fitting 5 folds for each of 4 candidates, totalling 20 fits

C:\Users\USER\anaconda3\Lib\site-packages\xgboost\training.py:200:
UserWarning: [20:37:30] WARNING: C:\actions-runner\_work\xgboost\
xgboost\src\learner.cc:782:
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)
```

```
GridSearchCV(cv=5,
             estimator=XGBClassifier(base_score=None, booster=None,
                                     callbacks=None,
colsample_bylevel=None,
                                     colsample_bynode=None,
                                     colsample_bytree=None,
device=None,
                                     early_stopping_rounds=None,
                                     enable_categorical=False,
                                     eval_metric='logloss',
feature_types=None,
                                     feature_weights=None, gamma=None,
                                     grow_policy=None,
importance_type=None,
                                     interaction_constraint...
                                     max_cat_threshold=None,
                                     max_cat_to_onehot=None,
                                     max_delta_step=None,
max_depth=None,
                                     max_leaves=None,
min_child_weight=None,
                                     missing=nan,
monotone_constraints=None,
                                     multi_strategy=None,
n_estimators=None,
                                     n_jobs=None,
num_parallel_tree=None, ...),
             n_jobs=-1,
             param_grid={'learning_rate': [0.1], 'max_depth': [3, 5],
                         'n_estimators': [100, 200]},
             scoring='f1_weighted', verbose=2)

#Printing the best parameters for the model

print("Best Parameters:", grid_search_XG_multi.best_params_)
print("Best CV Score:", grid_search_XG_multi.best_score_)

Best Parameters: {'learning_rate': 0.1, 'max_depth': 5,
'n_estimators': 200}
Best CV Score: 0.6088512391068199

#Making predictions with the best parameters and  model evaluation

best_XG_multi = grid_search_XG_multi.best_estimator_

ym_pred_XG_tuned_multi = best_XG_multi.predict(Xm_test_vec_multi)

cm_XG_tuned_multi =
confusion_matrix(ym_test_encoded_multi,ym_pred_XG_tuned_multi)
```

```
display_XG_tuned_multi = ConfusionMatrixDisplay(confusion_matrix =
cm_XG_tuned_multi, display_labels=["Negative emotions","Neutral
emotions", "Positive emotions"])
display_XG_tuned_multi.plot(cmap=plt.cm.Blues)
plt.show()
print(classification_report(ym_test_encoded_multi,
ym_pred_XG_tuned_multi))
```



	precision	recall	f1-score	support
0	0.62	0.12	0.19	113
1	0.67	0.87	0.75	929
2	0.63	0.42	0.51	589
accuracy			0.66	1631
macro avg	0.64	0.47	0.48	1631
weighted avg	0.65	0.66	0.63	1631

Confusion Matrix Interpretation

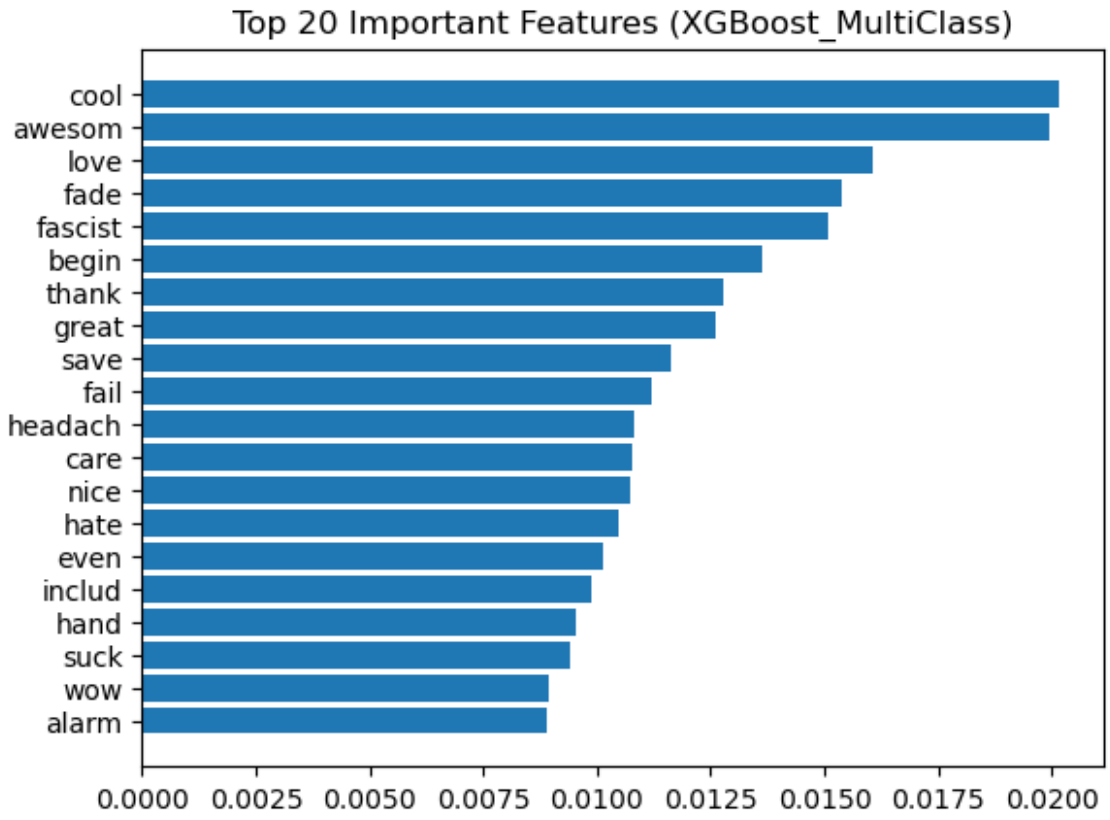
There was a reduction in precision for Negative emotions, an increase for Neutral and Positive emotions. In recall, there was an increase in Negative emotions, a reduction in Neutral emotions and an increase in Positive emotions. For f1-score, there was an increase for Negative emotions,

nochange forNeutral Emotions and an increase for Positive emotions. Generally, this model performed better than the base model for MultiClass classification hence making it the better model.

```
#Model interpretability for XG Boost Multi Class Classification

feature_importance_multi =pd.DataFrame({
    'feature':vectorize_multi.get_feature_names_out(),
    'importance': best_XG_multi.feature_importances_
}).sort_values('importance', ascending =False)

#Viewing the most important words
feature_importance_multi.head(20)
```

	feature	importance
1064	cool	0.011219
358	awesom	0.009377
2986	love	0.007953
1732	fade	0.007560
1759	fascist	0.006971
464	begin	0.006967
5034	thank	0.006752
2162	great	0.006562
4302	save	0.006325
1733	fail	0.006000
2285	headach	0.005990
747	care	0.005841
3391	nice	0.005750
2270	hate	0.005341
1649	even	0.005158
2520	includ	0.005060
2241	hand	0.005037
4820	suck	0.004877
5664	wow	0.004836
112	alarm	0.004545

```
#Vizualization of feature importance above

top_features_multi = feature_importance_multi.head(20)

plt.barh(top_features_multi["feature"], top_features["importance"])
plt.gca().invert_yaxis()
plt.title("Top 20 Important Features (XGBoost_MultiClass)")
plt.show()
```



Interpretation of Key features

Words like cool, awesome, love, great have been the best drivers in the positive emotions category. Words like hate, fail, suck have been the biggest drivers in the Negative emotion category. There are words that cannot be assigned to a specific category like even, hand, begin which should be thedrivers for the neutral category.

6. FINAL MODEL SELECTION

i. Binary Classification

Based on the models above and the insights on each one of them, the best model is the tuned XGBoost Model which achieved an improvement in recall and f1 score for Negative emotions. On the other hand, precision, recall and f1 score maintained at te level with its base model. This generally led to an improvement in the performance of the model hance making it the best model.

ii. Multiclass Classification

Multicalass classification was done on XGBoost only with a base and a tuned model. There was a reduction in precision for Negative emotions, an increase for Neutral and Positive emotions. In recall, there was an increase in Negative emotions, a reduction in Neutral emotions and an increase in Positive emotions. For f1-score, there was an increase for Negative emotions, nochange for Neutral Emotions and an increase for Positive emotions. Generally, this model performed better than the base model for MultiClass classification hence making it the better model.

7. RECOMMENDATIONS AND LIMITATIONS

BUSINESS RECOMMENDATIONS

1. The stakeholders should implement the best models in the classification of review sentiments
2. The stakeholders need to analys top features that drive negative emotions like headache, suck, hate and identify what exactly lead to such sentiments.
3. The stakeholders should focus more on negative sentiments and route them to customer support and product teams

EXPECTED IMPACT

1. Improved customer satisfaction.
2. Understading of product perception in customers

LIMITATIONS

1. Class imbalance in the dataset may influence the performance of the model as the data contains very few negatives and a majority of neutral sentiments.
2. The data was social media noisy and that may affect the performance of the model
3. The data may be from a specific region in the world and may not be fit for the stakeholder's region